

第 12 章

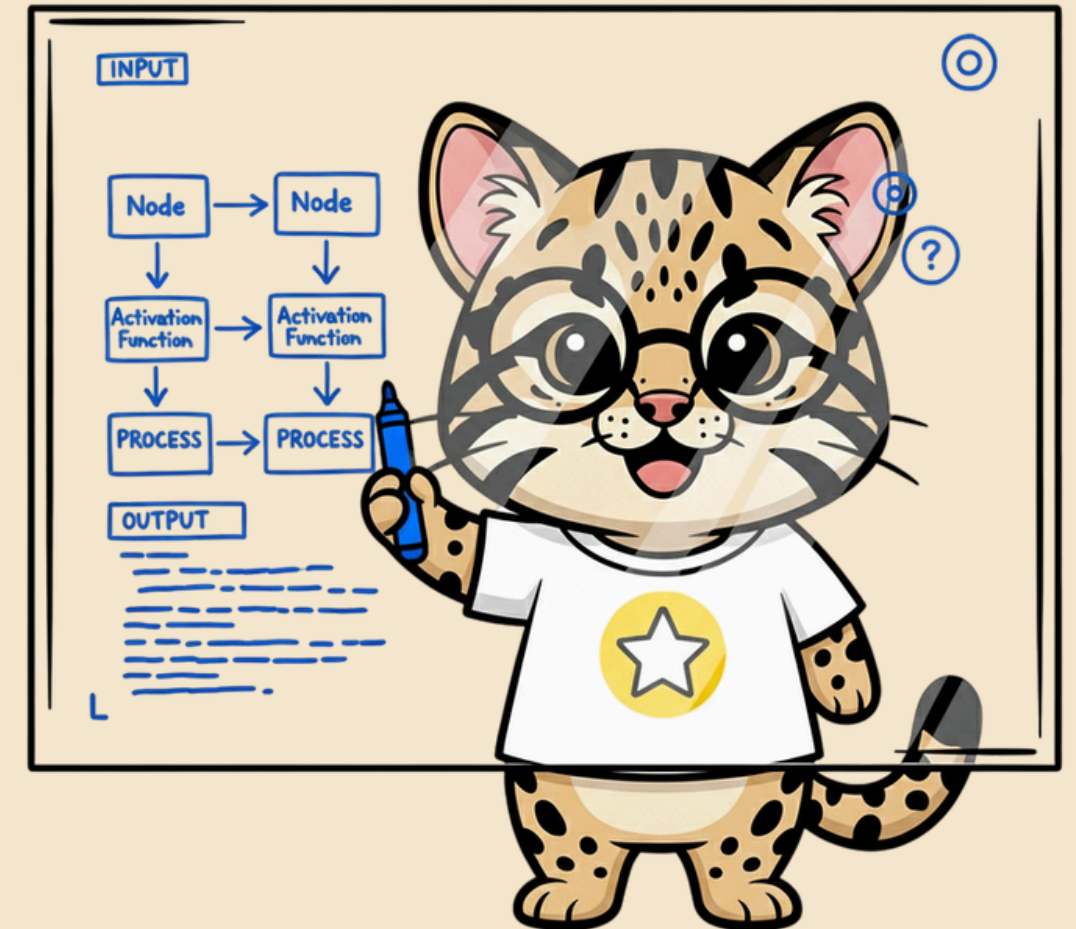
Fine-Tuning Generation Models

生成模型的微調：從只會接龍的 Base Model，
到聽得懂指令、答得符合偏好的助理模型

SFT

LoRA / QLoRA

Preference Tuning



今天要一起搞懂什麼？



1. 三個訓練步驟

Pretrain → SFT → Preference Tuning 的全局地圖



2. SFT 與 PEFT

Full fine-tuning、Adapters、LoRA 的取捨



3. QLoRA 實作

量化 + LoRA，動手微調 TinyLlama



4. 模型怎麼評估

詞級指標、Benchmark、人類評估、Goodhart's Law



5. Preference Tuning

Reward Model、PPO、DPO、ORPO 全部搞懂



打造高品質 LLM 的三個步驟



Base Model 只會「接龍」；*SFT* 教它「聽指令」；*Preference Tuning* 教它「答得更好」。

Base Model 不是不會回答，是「不懂要回答」

接龍成功的例子

The car is **red**

接龍失控的例子（沒有回答問題！）

What is 1 + 1?

2. What is 1+1+1?

3. What is 1+1+1+1? ...

- Base model 訓練目標只有「預測下一個字」，沒有「回答問題」這個行為模式
- 訓練語料裡，問題後面常接的是「更多問題」，不是「答案」
- SFT 的任務：教會模型在看到指令後，輸出「回應」而不是「接龍」

Supervised Fine-Tuning: 教它「聽指令」



- 訓練目標仍是 next-token prediction，差別在「標籤」變成使用者輸入之後該有的回應
- 「使用者輸入」本身某種意義上就是一種標籤 — 它定義了模型該生成什麼
- SFT 也能用於分類、摘要等任何有標籤任務，但最常見用途是把 Base Model 變成 Instruction / Chat 模型

「會回答」之後，還要「答得好」

What is an LLM ?

A *"It is a large language model."*

B *"An LLM is a neural network trained on massive text to predict the next token, giving it broad language and reasoning ability..."*

兩個答案都「正確」，但哪個更符合人類偏好？

- SFT 讓模型「會回答」，但不保證「答得好」
- 「好不好」其實是一種人類偏好，很難用單一規則寫死
- 第三步 Preference Tuning 就是要把這種偏好「教」給模型
- 關鍵字先劇透：Reward Model、PPO、DPO、ORPO

Full Fine-Tuning：最直觀，也最貴

預訓練 (Pretraining)

- 巨量文本
- 無標籤（自監督）
- 產出 Base Model



Unlabeled
Data

Full Fine-Tuning

- 更新「全部」參數
- 較小但有標籤的資料
- 效果好，但貴、慢、佔空間



Labeled
Data (Instructions)

任何有標籤資料都能拿來做 *full fine-tuning*，是學習領域專屬表示法的好方法 — 但成本是它最大的門檻，這也是後面 *PEFT* 出現的原因。

Instruction Data 長什麼樣子？

Question Answering

Instruction: What are large language models?

Output: Large language models (LLMs) are models that can generate human-like text by predicting the probability of a word given the previous words used in a sentence.

Sentiment Analysis

Instruction: Rate this review

Input: “This was a horrible place to eat!”

Output: This is a negative review.

同一份訓練資料可以混合非常多種任務類型，讓模型學會「依指令形式調整輸出」。

Full Fine-Tuning 太貴，怎麼辦？

- 訓練成本高：更新所有參數，GPU 時間 / 記憶體需求都很大
- 訓練速度慢：參數量越大，收斂所需的迭代成本越高
- 儲存成本高：每個任務都要保存一份「完整模型」的權重

Parameter-Efficient Fine-Tuning (PEFT)

只調整一小部分參數，或額外加上一小組新參數 — 大幅降低運算、時間、儲存成本，同時盡量逼近 full fine-tuning 的效果。

Adapters

LoRA / QLoRA

今天要看的兩個代表性技術 →

在 Transformer 裡插「小外掛」

Transformer Block

Self-Attention

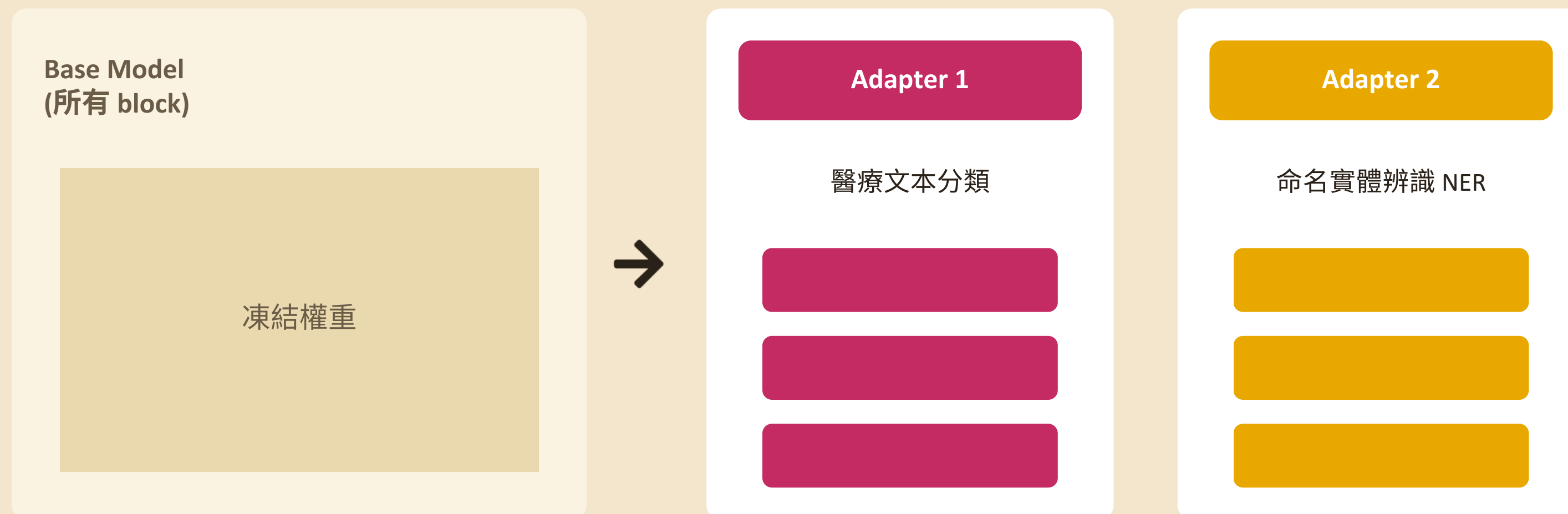
Adapter (元件 1)

Feedforward NN

Adapter (元件 2)

- 在 self-attention 與 feedforward 層之後，插入小型可訓練模組，其餘原始權重全部凍結
- 論文實測：只調 BERT 約 3.6% 參數量，GLUE 分數只跟 full fine-tuning 差 0.4%
- 代表大模型內有大量「冗餘」，不需要全部重新學

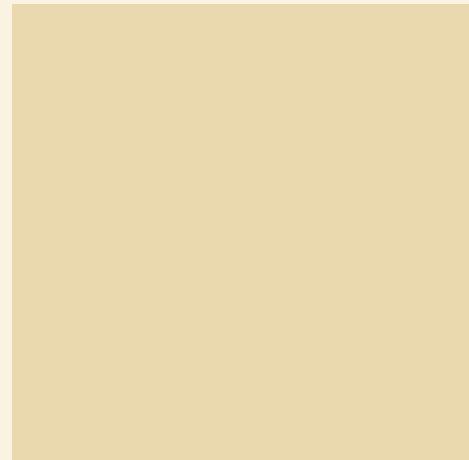
像模組一樣「插拔」的專家外掛



同一個 *base model*，可以依任務「插入」不同的 *adapter*，不需要為每個任務存一份完整模型。*AdapterHub* 就是分享這些 *adapter* 的平台。

Low-Rank Adaptation：拆出一個小子集來訓練

Base（凍結）



full fine-tuning
更新「全部」權重

LoRA

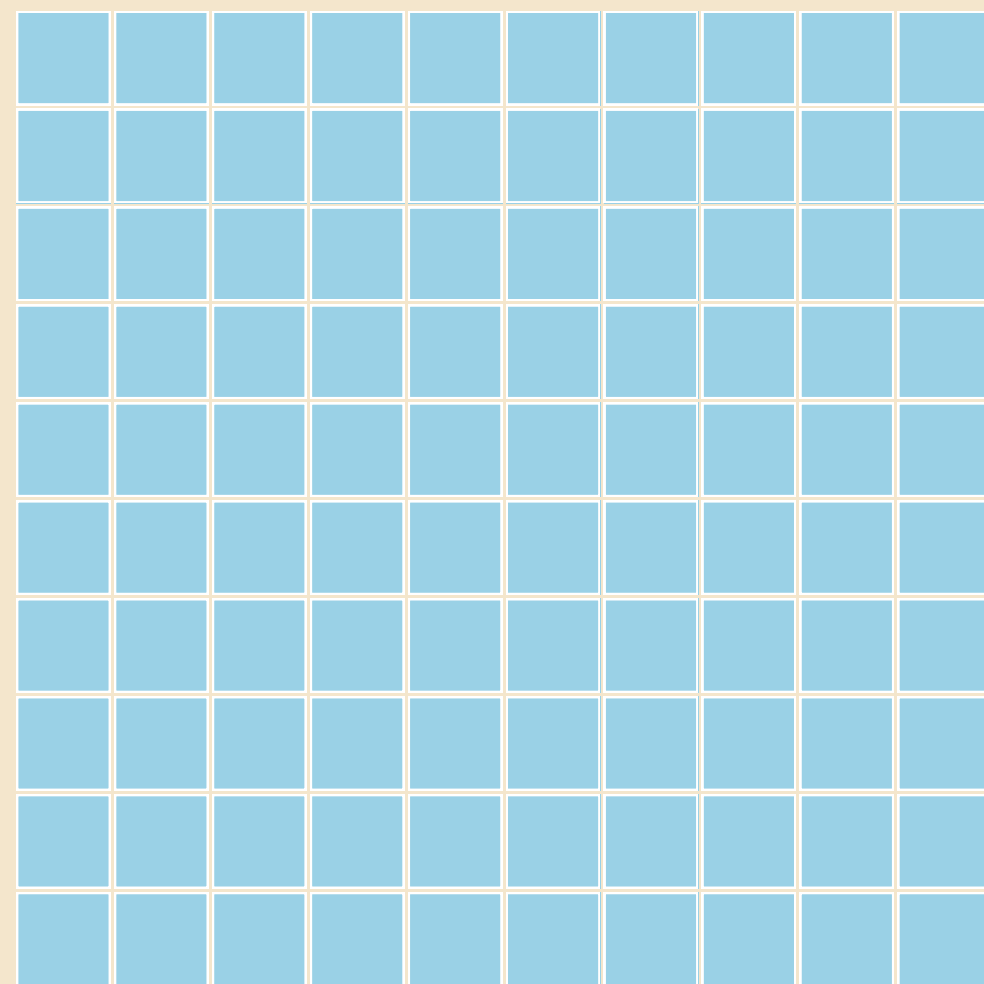


只訓練「拆出來的小矩陣」
原本大矩陣維持凍結
訓練完可分開保存、需要時再合併

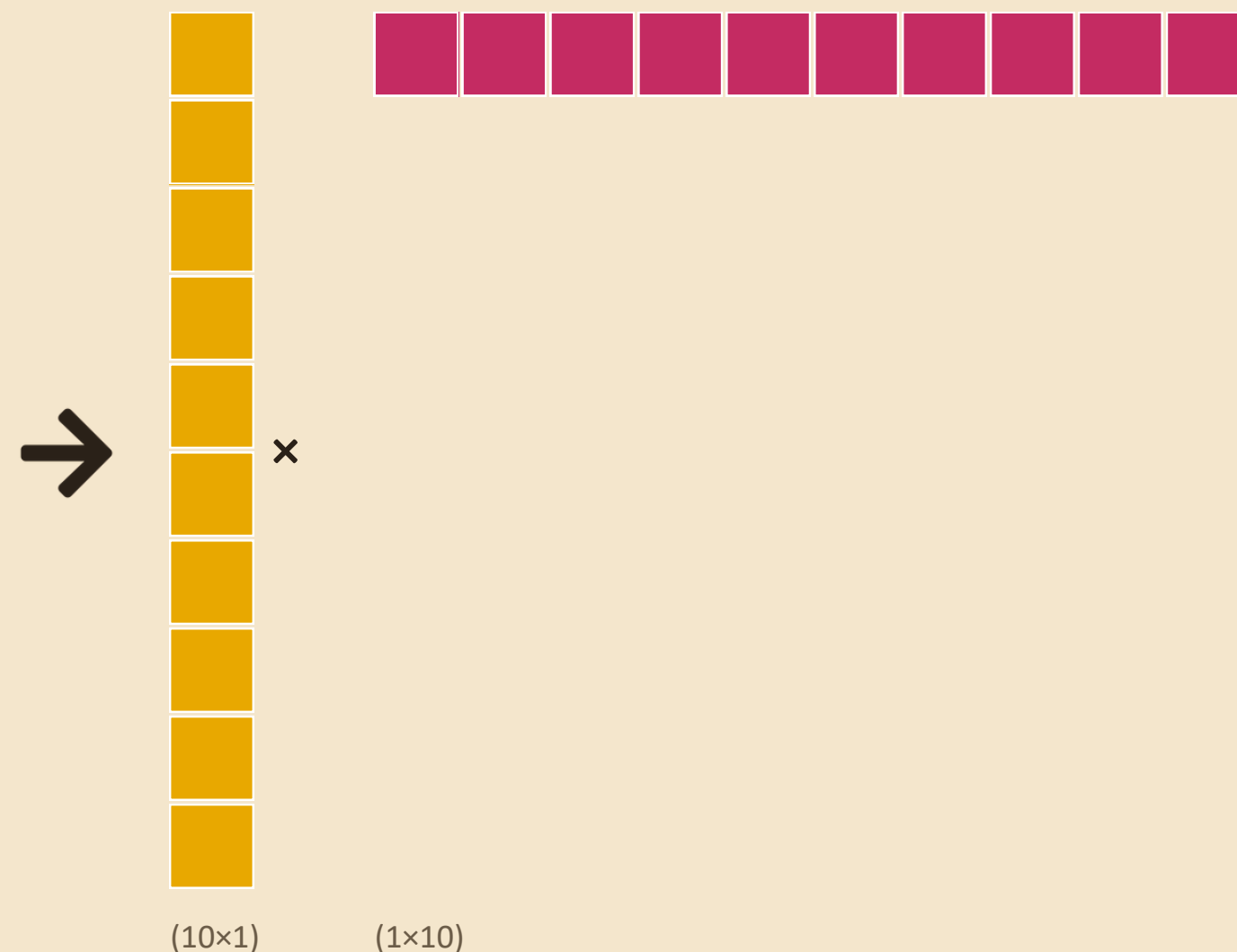
- 訓練速度快很多：只需更新一小組參數
- 同一個 base model，可以配上很多組不同任務的 LoRA 權重，切換很輕量
- 推論時，把「凍結權重」+「訓練好的小矩陣」合併使用

低秩分解：大矩陣 $\approx$ 兩個小矩陣相乘

原始權重矩陣 $10 \times 10 = 100$ 參數

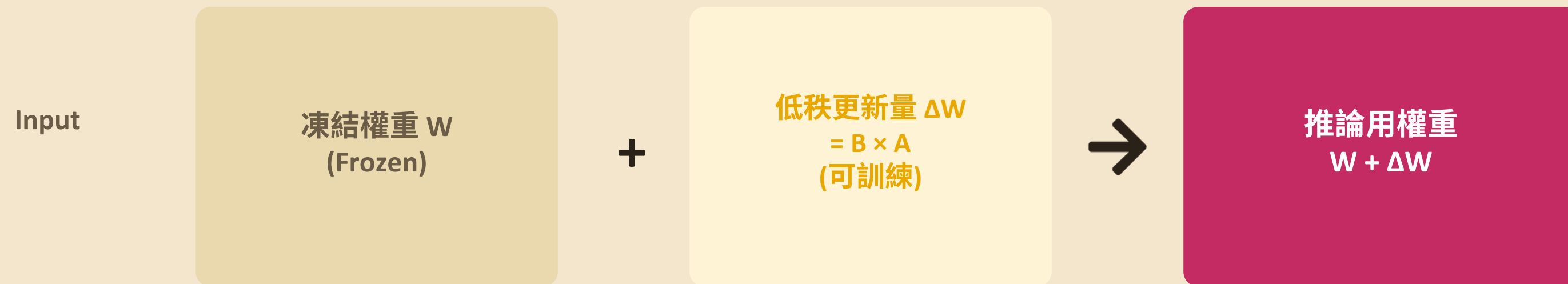


低秩分解 ($r=1$) $\rightarrow 20$ 參數



- GPT-3 規模：每層 $12,288 \times 12,288$ 矩陣 ≈ 1.5 億參數
- 用 $\text{rank}=8$ 分解，每層只需兩個 $12,288 \times 8$ 矩陣 ≈ 19.7 萬參數（約千分之一）
- 理論依據：LLM 具有「低內在維度」— 小 rank 也能有效逼近原本效果

訓練時到底更新了什麼？



- 訓練時：只針對新增的低秩矩陣做梯度更新，原始大矩陣 W 全程凍結
- 推論時：把「凍結權重」加上「訓練好的低秩更新量」合併使用
- 效果會不會打折扣？會，但幅度不大 — 因為語言模型具有「低內在維度」，小的自由度就足以逼近任務所需的調整
- 資料來源：《Intrinsic dimensionality explains the effectiveness of language model fine-tuning》

三個關鍵超參數

r (rank)

分解後小矩陣的維度，常見 4–64。越大表達力越強，但壓縮率越低。

lora_alpha

控制新知識與原模型知識的混合強度。常見經驗法則從 r 的一半到兩倍都有人用（本章範例其實用了 r 的一半：r=64, alpha=32），沒有絕對公式，需要實驗調整。

target_modules

決定套用 LoRA 的層，如 q_proj / k_proj / v_proj / o_proj、gate/up/down_proj。

沒有絕對正解 — 需要依資料集、任務、模型大小反覆實驗調整。

量化基礎：位元決定精度

Float 32-bit (高精度)

$\pi \approx 3.1415927$

32 bits / 數值

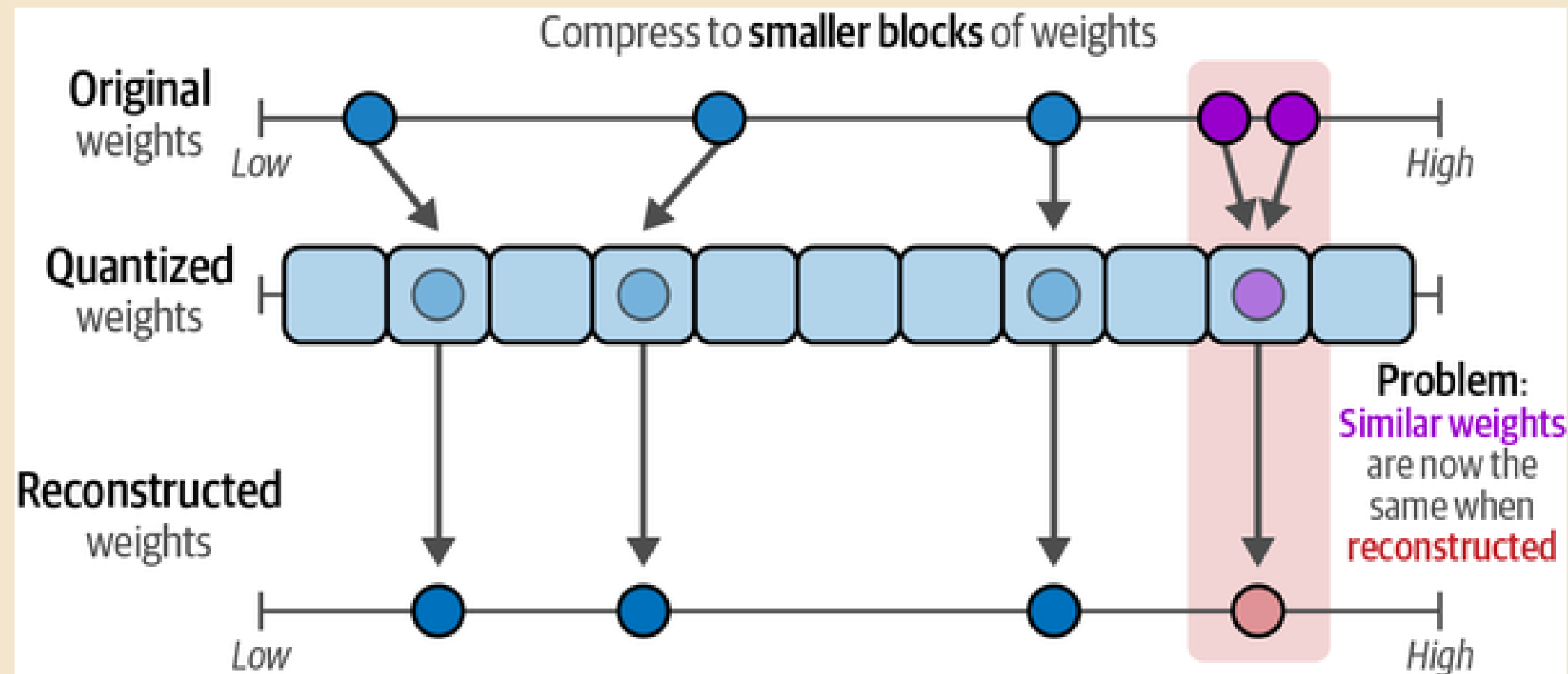
Float 16-bit (低精度)

$\pi \approx 3.141$

16 bits / 數值 — 記憶體減半

- 模型權重是數字，用幾個 bit 儲存就決定它的精度
- Bit 數降低 → 記憶體需求大幅減少，但數值精確度會變差
- 量化要處理的核心矛盾：怎麼在「省空間」與「保精度」之間取捨

直接量化的問題：資訊坍縮



這兩個原本不同的值
變得「一模一樣」！

- 把高精度數值直接對應到低精度區間，多個「原本不同但相近」的值會被壓成同一個值
- 權重之間的細微差異往往正是模型學到的知識所在，資訊因此流失

解法：Blockwise + 分布感知量化 (NF4)

① Blockwise Quantization

不對整個矩陣一次量化，而是切成小區塊分別處理 — 避免單一離群值拖垮整個區塊的精度分配。



② 分布感知分箱 (NF4)

神經網路權重多呈常態分布：密集區用更多、更細的量化區間；離群值區域區間放大，兼顧精度與效率。



- 再加上 double quantization（雙重量化）與 paged optimizer，進一步節省記憶體
- 效果：16-bit → 4-bit，VRAM 需求大幅下降，效能只小幅犧牲

實作總覽：用 QLoRA 教 TinyLlama 聽指令

- 模型：TinyLlama-1.1B（尚未學會聽指令的 base 版本）
- 資料：UltraChat 200k 的子集（3,000 筆對話）
- 先把資料整理成一致的 Chat Template，模型才學得起來

Chat Template

`<|user|>`

人類指令開始

`What is 1+1? </s>`

使用者輸入 + 結束符

`<|assistant|>`

模型生成開始

`The answer is 2! </s>`

模型回應 + 結束符



資料前處理：套用 Chat Template

```
def format_prompt(example):  
    chat = example["messages"]  
    prompt = template_tokenizer.apply_chat_template(  
        chat, tokenize=False)  
    return {"text": prompt}  
  
dataset = (  
    load_dataset("HuggingFaceH4/ultrachat_200k",  
        split="test_sft")  
    .shuffle(seed=42).select(range(3_000))  
)  
dataset = dataset.map(format_prompt)
```

- 用 tokenizer 的 `apply_chat_template` 統一套上 `<|user|> / <|assistant|>` 格式
- 取 UltraChat 200k 子集 3,000 筆，縮短訓練時間
- 格式化後存成 “text” 欄位，交給 SFTTrainer 使用

載入模型：4-bit 量化（QLoRA 的 Q）

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="float16",
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name, device_map="auto",
    quantization_config=bnb_config,
)
```

VRAM 需求對比

不量化 (16-bit) **~4 GB**

QLoRA (4-bit) **~1 GB**

實際訓練時，梯度與優化器狀態會再吃掉一些額外記憶體。

LoRA 設定與訓練參數

```
peft_config = LoraConfig(
    lora_alpha=32, lora_dropout=0.1,
    r=64, bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "k_proj", "q_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"]
)
```

```
training_arguments = TrainingArguments(
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    optim="paged_adamw_32bit",
    learning_rate=2e-4,
    lr_scheduler_type="cosine",
    num_train_epochs=1,
    fp16=True, gradient_checkpointing=True,
)
```

- cosine 學習率排程：先線性升溫，再依 cosine 曲線衰減
- num_train_epochs 保持低（此例為 1）避免過擬合、偏離原始知識過多
- paged_adamw_32bit：分頁式優化器，進一步節省顯示記憶體

開始訓練、合併權重、看結果

```
trainer = SFTTrainer(model=model,
                    train_dataset=dataset,
                    dataset_text_field="text",
                    args=training_arguments,
                    peft_config=peft_config)
trainer.train()
trainer.model.save_pretrained(
    "TinyLlama-1.1B-qlora")

# 合併回原始權重才能推論
model = AutoPeftModelForCausalLM.from_pretrained(
    "TinyLlama-1.1B-qlora")
merged_model = model.merge_and_unload()
```

微調前後對比

Base Model

胡亂接龍、生出更多問題，而不是回答

SFT 後模型 (Prompt: Tell me something about Large Language Models.)

"Large Language Models (LLMs) are artificial intelligence (AI) models that learn language and understand what it means to say things in a particular language..."

怎麼知道模型「好不好」？

- 生成模型應用場景太廣 — 數學解得好 $\neq$ 程式寫得好，很難用單一指標判斷
- 詞級指標（Word-Level Metrics）：比對生成 token 與參考文本

Perplexity（困惑度）

“When a measure becomes a ____” $\rightarrow$ 模型認為下一個字是 “target” 的機率有多高？機率越高，模型越不「意外」，代表它越熟悉這種語言型態。

ROUGE

BLEU

BERTScore

共同弱點：不管文字順不順、有沒有創意，甚至答案對不對，都測不出來。

公開 Benchmark 一覽

Benchmark	測什麼
MMLU	57 種任務的多工語言理解
GLUE	多種語言理解任務組合
TruthfulQA	模型會不會「一本正經說錯話」
GSM8k	小學程度數學應用題
HellaSwag	常識推理的多選題
HumanEval	程式能力（164 道程式題）

優點：標準化、方便跨模型比較 | 缺點：可能被「刷榜」overfitting，且部分 benchmark 需要數小時才能跑完。

LLM 當裁判、真人來評分

LLM-as-a-Judge

用另一個 LLM 評分，或做兩兩比較（pairwise），讓第三個 LLM 判斷哪個回答較好。優點：能評估開放式問題，且隨 LLM 進步同步成長。

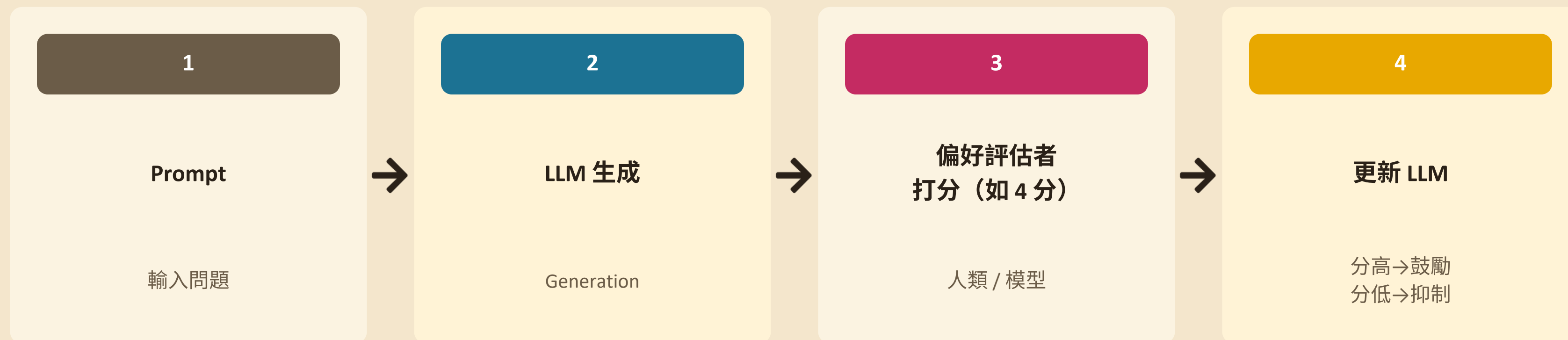
人工評估（Chatbot Arena）

真人跟兩個匿名模型互動、盲選較喜歡的回答，用西洋棋 Elo 系統累積投票算排名。書中認為這是評估的黃金標準。

“When a measure becomes a target, it ceases to be a good measure.”

— Goodhart's Law：只死命優化單一 benchmark，模型可能學會「應付考試」，而不是真的變好。

讓模型從「會回答」到「答得更好」



- 問題：每筆資料都靠真人打分，成本太高、規模有限
- 解法：把「打分」自動化 → 訓練一個 Reward Model 代替人類評分

Reward Model：讓模型自己學會打分



- 訓練資料：prompt + 一個 accepted 回應 + 一個 rejected 回應（不一定是好壞，也可能兩個都好，只是一個更好）
- 取得方式：讓 LLM 對同一 prompt 產生兩個版本，交由人類標註者選出偏好
- 進階：Llama 2 訓練兩個 RM，分別評「有幫助性」與「安全性」

PPO、DPO、ORPO 三種做法比一比

PPO

原始 *ChatGPT* 採用

用訓練好的 Reward Model 當獎勵，強化學習微調 LLM，並限制不要偏離太遠。

需同時維護 2 個模型，複雜、成本高

DPO

本章實作採用

不需要獨立 Reward Model，直接比較「凍結參考模型」與「可訓練模型」對 accepted / rejected 的機率變化。

更穩定、更簡單，效果不輸 PPO

ORPO

最新簡化方案

把 SFT 與 DPO 合併成單一訓練流程，一次到位。

不用分兩階段訓練，可搭配 QLoRA

資料前處理：格式化 accepted / rejected 配對

```
def format_prompt(example):
    system = "<|system|>\n" + example["system"] + "</s>\n"
    prompt = ("<|user|>\n" + example["input"]
              + "</s>\n<|assistant|>\n")
    chosen = example["chosen"] + "</s>\n"
    rejected = example["rejected"] + "</s>\n"
    return {
        "prompt": system + prompt,
        "chosen": chosen,
        "rejected": rejected,
    }

dpo_dataset = load_dataset(
    "argilla/distilabel-intel-orca-dpo-pairs",
    split="train")
dpo_dataset = dpo_dataset.filter(lambda r:
    r["status"] != "tie" and
    r["chosen_score"] >= 8 and
    not r["in_gsm8k_train"])
```

- 資料集：argilla/distilabel-intel-orca-dpo-pairs，內含 chosen / rejected 兩個回應與評分
- 過濾條件：排除平手（tie）、只留 chosen_score ≥ 8 、排除 GSM8k 題目，過濾後約 6,000 筆
- 格式化後產出三個欄位：prompt（含 system + user）、chosen、rejected，套用跟 SFT 一樣的 <|user|> / <|assistant|> 模板
- 跟 SFT 的資料格式最大差別：這次每筆資料要「同時」附上一個好回應跟一個壞回應，而不是只有一個標準答案

載入模型：接續 SFT 的 QLoRA adapter

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="float16",
    bnb_4bit_use_double_quant=True,
)

# 重新以 4-bit 量化載入「SFT 訓練好的」adapter
model = AutoPeftModelForCausalLM.from_pretrained(
    "TinyLlama-1.1B-qlora",
    device_map="auto",
    quantization_config=bnb_config,
)

# 在 SFT adapter 之上，再套一組新的 LoRA
# 準備訓練「DPO 用」的第二個 adapter
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, peft_config)
```

- 不是從 base model 重新開始，而是接著 Slide 23 存下來的 TinyLlama-1.1B-qlora (SFT adapter) 繼續訓練
- 一樣先做 4-bit 量化 (QLoRA 的 Q)，VRAM 需求維持在消費級 GPU 可負擔的範圍
- LoraConfig 沿用跟 SFT 相同的設定 (r=64、lora_alpha=32、target_modules 一致)，但這是「另一組獨立的」LoRA 權重
- 訓練完成後，SFT adapter 跟 DPO adapter 會是兩份分開保存的權重，推論前才依序合併

訓練設定：DPOConfig 與 DPOTrainer

```
training_arguments = DPOConfig(  
    output_dir="./results",  
    per_device_train_batch_size=2,  
    gradient_accumulation_steps=4,  
    optim="paged_adamw_32bit",  
    learning_rate=1e-5,  
    lr_scheduler_type="cosine",  
    max_steps=200,  
    fp16=True,  
    gradient_checkpointing=True,  
    warmup_ratio=0.1,  
)
```

```
dpo_trainer = DPOTrainer(  
    model,  
    args=training_arguments,  
    train_dataset=dpo_dataset,  
    tokenizer=tokenizer,  
    peft_config=peft_config,  
    beta=0.1,  
    max_prompt_length=512,  
    max_length=512,  
)  
  
dpo_trainer.train()
```

learning_rate 比 SFT 小十倍 (1e-5 vs 2e-4)，偏好對齊只需要「微調」行為，不宜跳太快；beta=0.1 控制可訓練模型跟凍結參考模型能偏離多少；max_steps=200 是課堂示範用的縮短版，不是完整訓練。

存檔、依序合併、比較 SFT 前後差異

```
dpo_trainer.model.save_pretrained(
    "TinyLlama-1.1B-dpo-qlora")

# 先合併 base + SFT adapter
sft_model = AutoPeftModelForCausalLM.from_pretrained(
    "TinyLlama-1.1B-qlora"
).merge_and_unload()

# 再把 DPO adapter 掛上 SFT 模型並合併
dpo_model = PeftModel.from_pretrained(
    sft_model,
    "TinyLlama-1.1B-dpo-qlora",
)
dpo_model = dpo_model.merge_and_unload()
```

- 合併順序不能顛倒：base model → 疊上 SFT adapter → 疊上 DPO adapter，一路往上疊
- 最終的 dpo_model 才是「聽得懂指令、又更符合人類偏好」的完整模型
- 用同一句 Prompt 「Tell me something about Large Language Models.」分別餵給 SFT 版跟 DPO 版，比較兩者回答的差異
- 本章示範只跑 200 steps，實務上要做完整 DPO 訓練，效果差異會更明顯

總結

微調 = 教會回答 + 教會答得更好

- SFT：讓 Base Model 從「接龍」變成「聽得懂指令」
- LoRA / QLoRA：用低秩分解 + 量化，把訓練成本壓到消費級 GPU 也能負擔
- Preference Tuning (PPO / DPO / ORPO)：讓模型「答得更符合人類偏好」
- 評估沒有單一完美指標 — 詞級指標、Benchmark、LLM 裁判、人工評估互補使用，並記得 Goodhart's Law

謝謝聆聽，Q&A 時間到！

<https://github.com/ai-twinkle/LLM-Book-Club>



預訓練語言模型進行微調（Fine-Tuning）的主要目的是什麼？

- ☐ A. 在模型測試過程中，消除對人工評估的需求
- ☐ B. 在更大的資料集上，從零開始完整重新訓練模型
- ☐ C. 確保模型在一般使用情境下，都能維持嚴格的 token 層級準確度
- ☒ D. 針對特定任務或使用者偏好調整模型，藉此提升其效能與對齊程度



在偏好微調（Preference-Tuning）技巧中，DPO 與 PPO 有何不同？

- ☐ A. PPO 比 DPO 更穩定
- ☐ B. DPO 使用語言模型本身作為評估的參考
- ☐ C. DPO 使用外部模型進行偏好評估
- ☐ D. PPO 不涉及 Reward Model

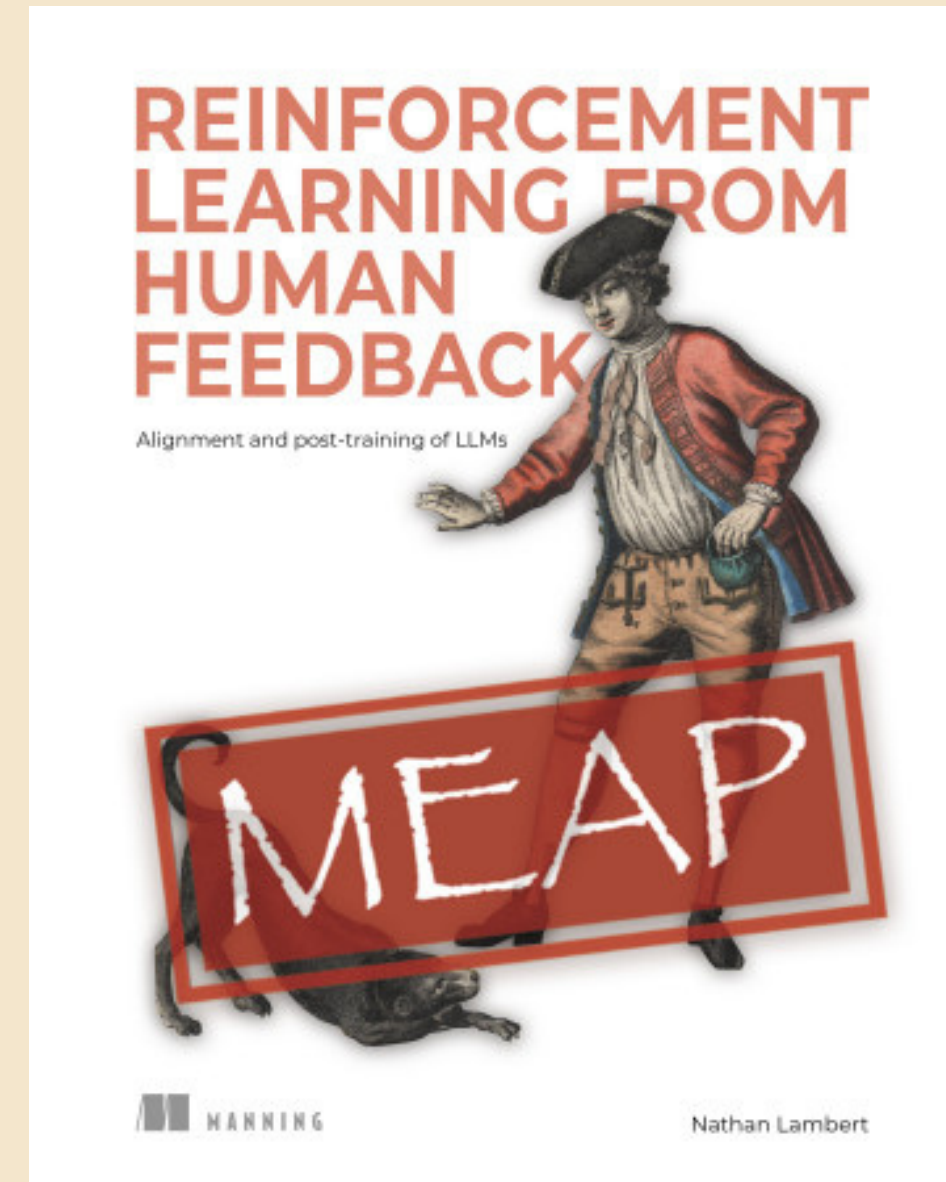
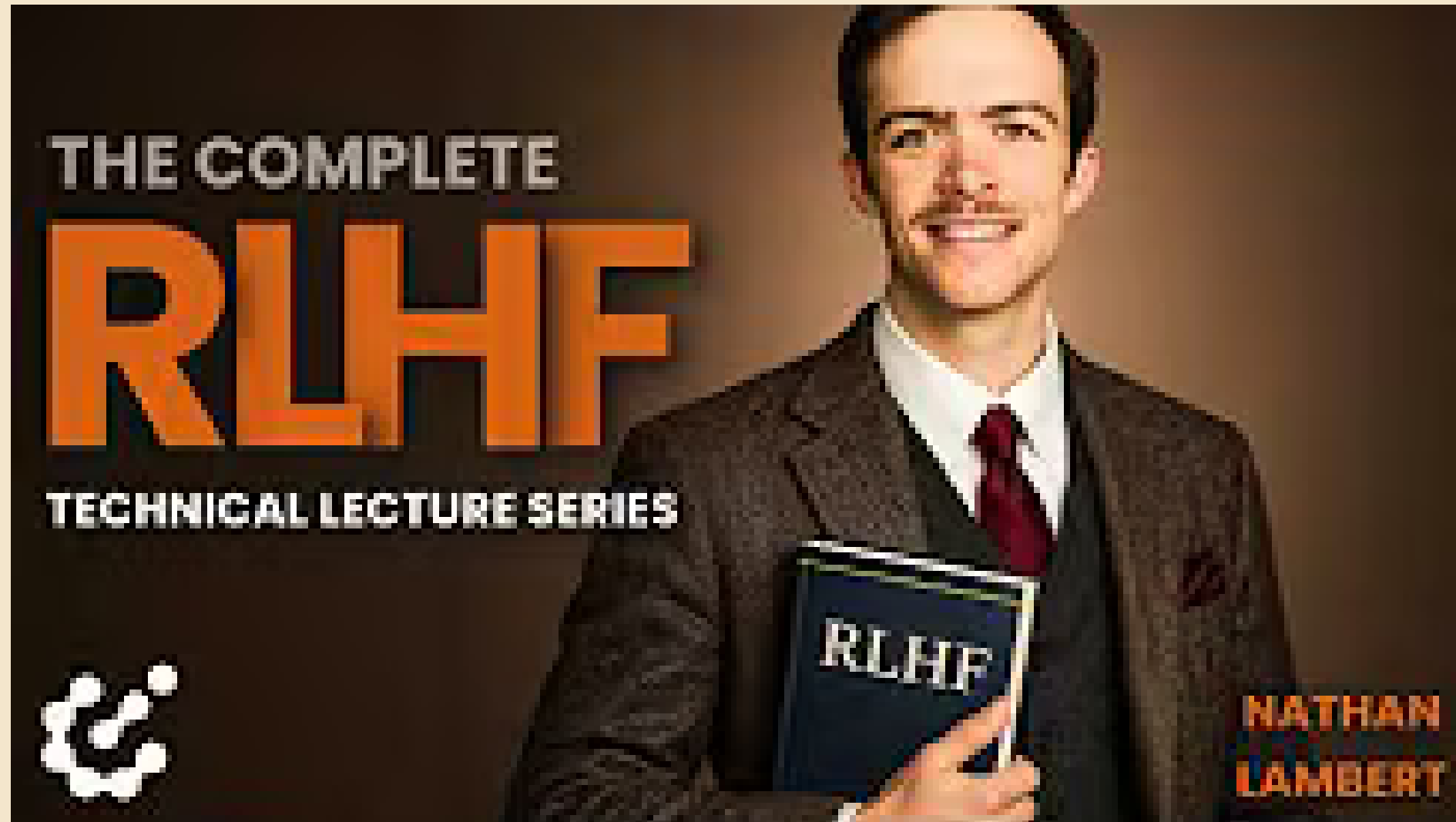
就資源使用而言，PEFT 與完整微調（Full Fine-Tuning）有何不同？

- ☐ A. PEFT 只更新一部分參數，因此資源消耗較低
- ☐ B. PEFT 使用更大的資料集，因此資源消耗更高
- ☐ C. PEFT 使用自動化評估方法，因此資源消耗更高
- ☐ D. PEFT 需要人工評估，增加資源需求

與完整微調相比，像 LoRA 這樣的 PEFT 方法有什麼關鍵優勢？

- ☐ A. 完全消除了對 PPO 或 DPO 等偏好微調技巧的需求
- ☐ B. 只專注在像 Perplexity 這類 token 層級指標上進行評估
- ☐ C. 只更新模型的一部分參數，降低運算與資源負擔
- ☐ D. 用較小的矩陣取代模型架構，藉此提升模型可解釋性

進階資源



<https://rlhfbook.com/>